

HW05: Funky Functions, Fun Exams

- **This homework must be completed with a partner! READ FIRST BEFORE STARTING!**

Learning Objectives

- Practice creating your first function
- Practice using return values from fruitful functions
- Review what you've learned in this course thus far, in preparation for Exam 1

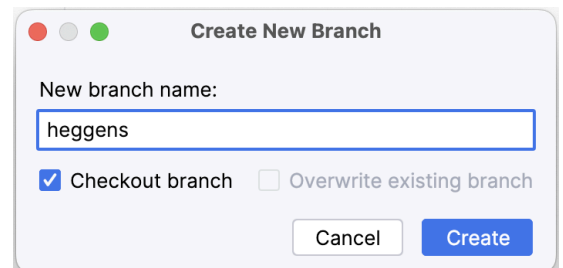
How to Begin

AFTER you've read through the assignment:

- Make a copy of this document by going to File >> Make a Copy...
- Share the copied document with all members of your team. You can share this document by hitting the blue button in the top right of the document, then entering the emails of all members in the input field.
- Change the file name of this document to **username1, username2 - HW05: Funky Functions, Fun Exams** (for example, **heggens, wilborned - HW05: Funky Functions, Fun Exams**). To do this, click the label in the top left corner of your browser.
- Next, go to the [Github classroom for HW05](#).
- You will be asked to create a team:
 - One of you will create the team.
 - Then, the other will join the team created by the first partner.
- Paste the link to your team's Github repo here:

Github Repo Link:	https://github.com/Berea-College-CSC-226/hw05-funky-functions-fun-exams-hw05_halil_aja_kornrumpfa.git
-------------------	---

- Open PyCharm.
- Click **Clone Repository** to clone the repository using the link above.
- Create a new git branch, named with just your username (e.g., **heggens** for me).



Exam 1 Preparation

Exam 1 is scheduled for Friday, February 28th, 2025. In preparation for the exam, we want you to begin thinking about reviewing for the exam. To do this, let's write some code!

Funky Functions

We've seen functions that do specific things: `draw_rectangle()`, `draw_square()`, etc. Most of these functions, when called, do their operations and simply end. Some functions, however, need to RETURN a value to the calling function. Some examples:

- `area(x, y)` - returns the area of a rectangle with sides of length `x` and `y`
- `circumference(r)` - returns the circumference of a circle with a radius `r`
- `fahrenheit(celsius)` - returns temperature in fahrenheit, given a celsius input
- `has_a_vowel(word)` - returns `True` if the word has a vowel in it, `False` otherwise

Go to **hw05_questions.md** and answer **SECTION 1**, then come back to this document to complete the rest of the assignment

How to Work With Your Partner

Unlike our usual homeworks, this one will a) require a partner, and b) we *encourage* you and your partner to divide and conquer this assignment.

First, check out the Issue Queue in Github. This is a feature of Github that allows you to keep notes about what problems, or issues, exist in your code. Each issue should be atomic, i.e., represents one change in the codebase that is requested. In the future when the issue is resolved and the new code is in the main branch, everything should still work, with one new feature.

There are ten issues, one for reading in our book. Your job is to:

1. Pick a role: **Function Designer** or **Unit Tester**.
2. Pick an issue and assign yourself to the issue. Do that here:
<https://github.com/Berea-College-CSC-226/hw05-main/issues>.
 - a. Pick an issue with *no more than 1* person from your section already assigned. Make a comment to indicate you'd like to take this issue. Use the following format:

```
SECTION [A or B] - [Function Designer/Unit Tester] - [First name]
```

Here's an example:

```
SECTION A - Unit Tester - Scott
```
 - b. Make note of the person who is assigned the matching issue for your chapter (there is a function design and unit tester for each). You will be working with this person to complete this assignment. If no one has picked the issue yet, just wait for them to do Step C.
 - c. If there *is* a person from your section assigned, reach out to this person *in Slack* to begin collaborating.

NOTE: This should be done before class on Friday!

3. As you both work, **you should be discussing back and forth design choices you are making** as you code, as they will impact each other's implementation. This communication can happen in-person (best), via Slack (good), through Github (okay), or via another means that is agreed upon between you and your partner (good only if agreed upon and *followed*).

Go to **hw05_questions.md** and answer the **SECTION 2** questions. Then, return to this document to continue the assignment.

Roles and Goals

1. **Function Designer:** Create a program that will satisfy the following:
 - a. Ask the user a multiple choice question related to your chapter.
 - b. Use a combination of `print` and `input` statements to format it nicely!
 - c. Use a fruitful function to:
 - i. Take in the user's response as an input parameter.
 - ii. Checks their answer and returns appropriately helpful feedback (i.e., if the answer is right, give them more information; if the answer is wrong, give them some help towards the right answer). Return this feedback as a string.
2. **Unit Tester:** Create a test suite that will satisfy the following:
 - a. Test all combinations of right and wrong answers for the fruitful function created by the Function Designer.
 - b. Tests at least four invalid inputs:
 - i. Two of which are of the right type of data, but not a possible answer
 - ii. Two of which are of the wrong type of data. They can be commented out, since they likely throw a runtime error.

Instructions

1. **Function Designers: ~ANT.**
 - a. You should be on a branch named *your username*. If not, create the branch now. | COMPLETE.
 - b. Create a new Python file called **chapter_X_function.py**, replacing the X with your chapter number. In this program, create a fruitful function that performs the operations described above.
 - c. Call your new fruitful function inside a `def main()` to prove it works.
 - d. Call the `main()` function inside an `if __name__ == "__main__":` statement to only run it when the program runs (and not when the test suite is used).
 - e. Commit and push your files to your repo when this file is complete. Remember to check that you are pushing to your branch, **not main!**
2. **Unit Testers: ~ARB.**
 - a. You should be on a branch named *your username* (**not** the same branch as your partner). If not, create the branch now.
 - b. Create a new file to hold your test suite called **chapter_X_suite.py**.
 - c. In the file, create a test suite that tests the function created by the Function Designer (if there are more than one, pick one that is not already being tested; again, communicate!). Use the provided example test suite and T05 as a guide for creating a useful test suite. Your tests should include:
 - i. An `import` statement which will import the Function Designer's file. This will likely give you a syntax error statement until you merge that file from your partner's branch into your branch (later).
 - ii. One test per valid input, that checks for results you expect.

HW05: Funky Functions, Fun Exams

- iii. At least two tests that check inputs that you DON'T expect (e.g., putting in "£" if there is no option £).

 **NOTE: These tests do not necessarily have to pass!**

- iv. At least two tests that test inputs that are of the wrong type to your function (e.g., putting in a floating point value instead of a string). Comment these out to avoid runtime errors.

 **NOTE: These tests do not necessarily have to pass!**

- d. Commit and push your files to your repo when this file is complete. Remember to check that you are pushing to your branch, **not main!**
3. Make sure you've followed our good coding practices as we've discussed in class (i.e. header block, commenting, docstrings, scoping of variables, etc.).

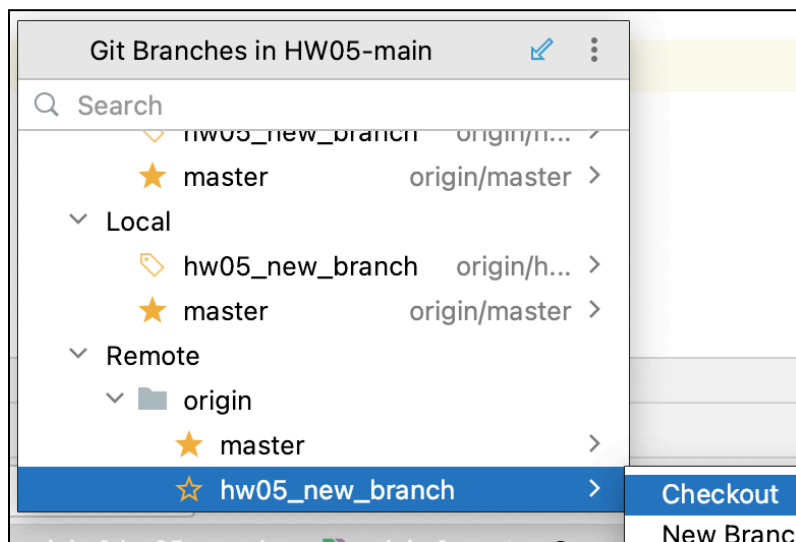
Bringing It All Together

When both codes above have been created, you'll need to synchronize your local repositories with the GitHub remote (because each of you is missing each other's file). We will assume the Function Designer will go first (it technically does not matter who goes first, but I would *not* suggest doing each step at the same time!).

Step 1: Merge the fruitful function

Function Designer: Commit and Push your code, then go to GitHub and issue a pull request for your branch to be merged into main. That's it!

Unit Tester: Review the pull request(s). First, look for obvious problems just by reading the code: syntax errors, code outside of functions, missing docstrings, etc. Leave them comments for things you think should be fixed. If you believe their code is good to go, open PyCharm and *test their code* by going to **Git >> Update Project...** and then **Git >> Branches >> Remote >> origin >> *branch_name* >> Checkout:**



Run their code a few times to make sure their fruitful function works as expected. If you think there are problems, discuss them with the Function Designer and fix the problems together, followed by commits and pushes by the Function Designer (which updates the Pull Request automatically). **Remember, communication is key!**

NOTE: You are NOT testing it with the test suite YET!

Once the **Unit Tester** is satisfied the fruitful function works properly, the Unit Tester merges the Pull Request which officially makes the Function Designer's code part of the main branch!

A green rectangular button with the text "Merge pull request" in white, followed by a small white downward-pointing triangle.

Step 2: Merge the test suite

Now that the Unit Tester has approved the Function Designer's work, repeat the same process for the Unit Tester's code. The Unit Tester will integrate the new main branch with the new file, test it with their unit tests, then create a new pull request in Github. Finally, the Function Designer takes over the review process:

1. (Unit Tester) Update PyCharm with everything in Github: **Git >> Update Project**
2. Switch to the Unit Tester's branch: **Git >> Branches >> Remote >> origin >> *branch_name* >> Checkout**
3. Merge main into your branch using **Git >> Merge >> origin/main**. You should see the fruitful function file and code in your main branch now!
4. Resolve any merge conflicts. Since you worked in separate files, there should be no conflicts!
5. Run their test suite and confirm that it works with your fruitful function, testing various inputs and the resulting `return` values. Fix any problems.
6. Communicate with the Unit Tester about any necessary fixes, which **they** will commit and push to update the Pull Request. Update your project, and keep testing until everything works as expected.
7. Once satisfied, accept by merging the Pull Request!

A green rectangular button with the text "Merge pull request" in white, followed by a small white downward-pointing triangle.

Once both pull requests have been merged, switch to the main branch, and use the Git menu in PyCharm to do a **Pull** on the main branch. This should bring down all of the code into your local repository. Test both pieces of code to ensure they still work together in the **main** branch!

Go to **hw05_questions.md** and answer the **SECTION 3** questions. Then, return to this document to continue the assignment.

Submission Instructions

Follow the [submission instructions](#) to submit your work by next Wednesday at 11:55PM.